

Week 4 Assignment: Training PPO on the Trading Environment

Overview

In this assignment you will do four things:

1. **Train PPO on your Week 3 ToyTradingEnv** and compare it to the random baseline from last week.
2. **Extend the trading environment** with a price-history window and a transaction cost.
3. **Evaluate the trained agent properly** — plotting cumulative P&L and comparing against a random agent and a buy-and-hold baseline.
4. **Reflect in writing** on what the agent learned (or did not learn) and why.

Setup

All packages from Week 3 are still required. Confirm they import before starting:

```
import gymnasium as gym
import numpy as np
import torch
from stable_baselines3 import PPO
import matplotlib.pyplot as plt
```

Copy `week3_trading_env.py` into your Week 4 folder (or import from it). That is your starting point.

Project Structure

By the end of Week 4, your folder should look like:

```
project_root/
├── week3_trading_env.py           (from Week 3)
├── week4_train_ppo_trading.py     (Task 1)
├── week4_trading_env_v2.py       (Task 2)
├── week4_evaluate.py             (Task 3)
├── week4_notes.md                (Task 4)
├── plots/
│   ├── reinforce_vs_ppo.png      (from Week 3)
│   ├── ppo_trading_rewards.png   (Task 1)
│   └── cumulative_pnl.png        (Task 3)
```

Task 1: Train PPO on ToyTradingEnv (Core)

If you completed the optional extension in Week 3, you already have a head start — this task makes that the required starting point.

Create `week4_train_ppo_trading.py` using this skeleton:

```

# week4_train_ppo_trading.py

import numpy as np
import matplotlib.pyplot as plt
import os
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import BaseCallback
from week3_trading_env import ToyTradingEnv

class EpisodeRewardLogger(BaseCallback):
    """Records the total reward of each completed episode."""
    def __init__(self):
        super().__init__()
        self.episode_rewards = []
        self._current_reward = 0.0

    def _on_step(self) -> bool:
        self._current_reward += self.locals["rewards"][0]
        if self.locals["dones"][0]:
            self.episode_rewards.append(self._current_reward)
            self._current_reward = 0.0
        return True

def train_ppo_on_trading(total_timesteps=100_000):
    env = ToyTradingEnv()
    callback = EpisodeRewardLogger()

    model = PPO(
        "MlpPolicy",
        env,
        verbose=0,
        learning_rate=3e-4,
        n_steps=512,      # TODO: add your own comment explaining this
        batch_size=64,    # TODO: add your own comment explaining this
        gamma=0.99,
    )
    model.learn(total_timesteps=total_timesteps, callback=callback)
    env.close()
    return model, callback.episode_rewards

def moving_average(x, window=20):
    if len(x) < window:
        return np.array(x)
    return np.convolve(x, np.ones(window) / window, mode="valid")

def plot_rewards(episode_rewards,
                 filename="plots/ppo_trading_rewards.png"):
    os.makedirs("plots", exist_ok=True)
    rewards = np.array(episode_rewards)

```

```

plt.figure(figsize=(9, 4))
plt.plot(rewards, alpha=0.25, color="steelblue",
        label="Episode reward")
plt.plot(moving_average(rewards), color="steelblue",
        label="20-ep moving avg")
plt.axhline(0, color="gray", linestyle="--",
        linewidth=0.8, label="Zero")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("PPO on ToyTradingEnv")
plt.legend()
plt.tight_layout()
plt.savefig(filename)
plt.close()
print(f"Saved to {filename}")

if __name__ == "__main__":
    model, rewards = train_ppo_on_trading(total_timesteps=100_000)
    plot_rewards(rewards)
    np.save("ppo_trading_returns.npy", np.array(rewards))
    print(f"Final 20-episode average: {np.mean(rewards[-20:]).2f}")

```

What You Must Do

1. Read through the skeleton and add comments to `n_steps` and `batch_size` in your own words — what does each parameter control in the PPO update cycle?
2. Run the script for 100 000 timesteps. The reward array is saved by the last line of `__main__` — confirm `ppo_trading_returns.npy` exists after training.
3. Look at the reward curve in `plots/ppo_trading_rewards.png`. Ask yourself (you will write this up in Task 4):
 - Does the moving average trend upward, stay flat, or oscillate around zero?
 - Is this what you expected, given that the environment uses a random walk?
4. Compare the final 20-episode average reward of the PPO agent to the random agent's average from Week 3 Task 4. Print both numbers side by side.

Task 2: Extend the Trading Environment (Core)

The Week 3 state was only two numbers: `[last_price_return, current_position]`. Now you will add a **price history window** and a **transaction cost**.

Create `week4.trading_env_v2.py`:

```

# week4_trading_env_v2.py

import gymnasium as gym
from gymnasium import spaces
import numpy as np
from collections import deque

```

```

class TradingEnvV2(gym.Env):
    """
    Extended trading environment with:
    - A window of the last 'history_len' price returns in the state.
    - A transaction cost subtracted from the reward when position
      changes.

    Observation (history_len + 1 dimensional):
        [r_{t-history_len+1}, ..., r_t, current_position]

    Actions (discrete):
        0 = Go Long
        1 = Hold
        2 = Go Short

    Reward:
        position * price_return * 100
        - transaction_cost * |new_position - old_position|
    """

    def __init__(
        self,
        episode_length=100,
        initial_price=100.0,
        daily_vol=0.01,
        history_len=5,
        transaction_cost=0.1,
    ):
        super().__init__()
        self.episode_length = episode_length
        self.initial_price = initial_price
        self.daily_vol = daily_vol
        self.history_len = history_len
        self.transaction_cost = transaction_cost

        obs_dim = history_len + 1 # history returns + current position
        self.observation_space = spaces.Box(
            low=np.full(obs_dim, -np.inf, dtype=np.float32),
            high=np.full(obs_dim, np.inf, dtype=np.float32),
        )
        self.action_space = spaces.Discrete(3)

        self.price = None
        self.position = None
        self.step_count = None
        self._price_history = None

    # ----- #
    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.price = self.initial_price
        self.position = 0.0

```

```

self.step_count      = 0
self._price_history = deque(
    [0.0] * self.history_len, maxlen=self.history_len
)
return self._get_obs(last_return=0.0), {}

# ----- #
def step(self, action):
    # Price dynamics
    price_return = float(
        self.np_random.normal(0.0, self.daily_vol)
    )
    self.price *= (1.0 + price_return)
    self._price_history.append(price_return)

    # Map action to new position
    action_to_position = {0: 1.0, 1: self.position, 2: -1.0}
    new_position = action_to_position[int(action)]

    # Reward: P&L from old position minus transaction cost
    pnl = self.position * price_return * 100.0
    cost = self.transaction_cost * abs(new_position - self.position)
    reward = pnl - cost

    # TODO: add a comment here explaining why cost uses
    # (new_position - self.position) and when it equals zero

    self.position = new_position
    self.step_count += 1
    terminated = self.step_count >= self.episode_length
    truncated = False

    return self._get_obs(price_return), reward, terminated,
        truncated, {}

# ----- #
def _get_obs(self, last_return):
    history = np.array(list(self._price_history), dtype=np.float32)
    return np.append(history, self.position).astype(np.float32)

# ----- #
def render(self):
    print(
        f"Step {self.step_count:3d} | "
        f"Price {self.price:8.2f} | "
        f"Position {self.position:+.0f}"
    )

```

What You Must Do

1. Read through the skeleton and fill in the one TODO comment in `step()` in your own words.
2. Run a sanity check on the new environment:

```
from gymnasium.utils.env_checker import check_env
from week4_trading_env_v2 import TradingEnvV2
check_env(TradingEnvV2(), warn=True)
```

Fix any warnings before continuing.

3. At the bottom of the file (or in a short script), run 3 episodes with a random agent on `TradingEnvV2` and print the total reward per episode. Confirm the rewards are finite numbers (not NaN or inf).
4. Compare the observation shape from `TradingEnvV2` to the shape from `ToyTradingEnv`. Print both. What changed and why?

Task 3: Evaluate the Agent Properly (Core)

Now train PPO on `TradingEnvV2` and compare it to two baselines: a random agent and a buy-and-hold agent. Create `week4.evaluate.py`:

```
# week4_evaluate.py

import numpy as np
import matplotlib.pyplot as plt
import os
from stable_baselines3 import PPO
from week4_trading_env_v2 import TradingEnvV2

def run_episode(env, model=None):
    """Run one episode. model=None means random agent."""
    obs, _ = env.reset()
    rewards = []
    done = False
    while not done:
        if model is None:
            action = env.action_space.sample()
        else:
            action, _ = model.predict(obs, deterministic=True)
        obs, reward, terminated, truncated, _ = env.step(action)
        rewards.append(reward)
        done = terminated or truncated
    return rewards

def buy_and_hold_episode(env):
    """Always stay long (action=0). Returns per-step rewards."""
    obs, _ = env.reset()
    rewards = []
    done = False
    while not done:
        obs, reward, terminated, truncated, _ = env.step(0)
        rewards.append(reward)
        done = terminated or truncated
```

```

    return rewards

def average_cumulative_pnl(env, n_episodes=50,
                           model=None, strategy="random"):
    """Return the mean cumulative P&L curve over n_episodes."""
    all_cumulative = []
    for _ in range(n_episodes):
        if strategy == "buy_and_hold":
            step_rewards = buy_and_hold_episode(env)
        else:
            step_rewards = run_episode(env, model=model)
        all_cumulative.append(np.cumsum(step_rewards))
    min_len = min(len(c) for c in all_cumulative)
    stacked = np.array([c[:min_len] for c in all_cumulative])
    return stacked.mean(axis=0)

def plot_pnl(pnl_trained, pnl_random, pnl_bnh,
             filename="plots/cumulative_pnl.png"):
    os.makedirs("plots", exist_ok=True)
    steps = np.arange(len(pnl_trained))
    plt.figure(figsize=(9, 4))
    plt.plot(steps, pnl_trained, color="steelblue",
             label="PPO (trained)")
    plt.plot(steps, pnl_random, color="darkorange",
             label="Random agent")
    plt.plot(steps, pnl_bnh, color="green",
             label="Buy and hold")
    plt.axhline(0, color="gray", linestyle="--", linewidth=0.8)
    plt.xlabel("Step within episode")
    plt.ylabel("Cumulative P&L (mean over 50 episodes)")
    plt.title("Agent comparison on TradingEnvV2")
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename)
    plt.close()
    print(f"Saved to {filename}")

if __name__ == "__main__":
    env = TradingEnvV2()

    print("Training PPO on TradingEnvV2 ...")
    model = PPO(
        "MlpPolicy", env, verbose=0,
        learning_rate=3e-4, n_steps=512,
        batch_size=64, gamma=0.99,
    )
    model.learn(total_timesteps=100_000)
    print("Training complete.")

    env_eval = TradingEnvV2()
    pnl_trained = average_cumulative_pnl(

```

```

    env_eval, n_episodes=50, model=model, strategy="trained")
pnl_random = average_cumulative_pnl(
    env_eval, n_episodes=50, model=None, strategy="random")
pnl_bnh = average_cumulative_pnl(
    env_eval, n_episodes=50, model=None, strategy="buy_and_hold")

print(f"Final cumulative P&L (mean over 50 episodes):")
print(f"  PPO trained: {pnl_trained[-1]:.2f}")
print(f"  Random agent: {pnl_random[-1]:.2f}")
print(f"  Buy and hold: {pnl_bnh[-1]:.2f}")

plot_pnl(pnl_trained, pnl_random, pnl_bnh)

```

What You Must Do

1. Run `week4_evaluate.py` and save the plot `plots/cumulative_pnl.png`.
2. Look at the final cumulative P&L numbers for all three strategies. Do they cluster near zero? Are any of them surprisingly positive or negative? Write down your observation — you will need it for Task 4.
3. Run the same evaluation with `transaction_cost=0.0`. Do the three curves separate more or less? Why? You do not need a separate file — create a second `TradingEnvV2(transaction_cost=0.0)` and call the functions again.
4. Read through `run_episode` and `buy_and_hold_episode`. Add comments in your own words explaining:
 - Why `model.predict(obs, deterministic=True)` is used for evaluation instead of `deterministic=False`.
 - What `np.cumsum` does to the list of per-step rewards.

Task 4: Written Reflection (`week4_notes.md`)

Create `week4_notes.md` and answer each question in 2–4 sentences.

On training PPO on the trading environment (Task 1):

1. Describe the reward curve from Task 1. Does the moving average trend upward over 100 000 timesteps, or does it stay flat near zero? Was this what you expected? Why or why not?
2. Compare the PPO agent's final 20-episode average reward to the random agent's average from Week 3 Task 4. Is there a meaningful difference? What does this tell you about whether PPO found a useful policy?

On the extended environment (Task 2):

3. The transaction cost discourages the agent from changing position every step. Describe in your own words what type of policy would minimise transaction costs while still earning reward. Is that a good trading strategy?

4. You added `history_len=5` past returns to the state. On a true random walk, past returns contain no information about future returns. Given that, why might it still be worth including them in the state?

On evaluation (Task 3):

5. Look at your `plots/cumulative_pnl.png`. Do the three strategies (PPO, random, buy-and-hold) end up near the same cumulative P&L, or does one clearly outperform the others? Explain what this tells you about the environment.
6. When you removed the transaction cost (`transaction_cost=0.0`), what happened to the three curves? Did the PPO agent perform better, worse, or the same relative to the random baseline?
7. In your own words, explain why `deterministic=True` in `model.predict` during evaluation is different from `deterministic=False` during training. When would you want each?

Big picture:

8. After four weeks of RL (REINFORCE → PPO → trading environment → evaluation), what do you think is the hardest part of applying RL to financial markets? Is it the algorithm, the environment design, the reward, or something else? Justify your answer in 3–5 sentences.
9. List 3 things you would want to add to `TradingEnvV2` before you would trust an agent trained on it to trade real money. For each, say whether it changes the state, the action space, the reward, or the episode structure.
10. Write down **3 questions** about trading environments or RL for finance you want to discuss with your mentor.

Deliverables

By the end of Week 4, you should have:

- **Code:** `week4_train_ppo_trading.py`, `week4_trading_env_v2.py`, `week4_evaluate.py`.
- **Notes:** `week4_notes.md` with answers to all 10 questions.
- **Plots:** `plots/ppo_trading_rewards.png` and `plots/cumulative_pnl.png`.

During your mentor meeting, walk through:

- The reward curve from Task 1 — does it show learning or noise?
- The cumulative P&L plot — what story does it tell about the three strategies?
- Your answer to Question 8 — what is the hardest part of RL for finance?
- The 3 questions from Question 10 in your notes.

Common Issues

“My PPO agent is learning but the P&L is still near zero.”

That is correct behaviour on a random walk. “Learning” in the RL sense means the reward curve

stabilises — not that the agent beats an efficient market. Both things can be true at once.

check_env raises a warning about observation bounds.

If you used `-np.inf / np.inf` in your `Box`, some versions of Gymnasium warn about this. It is not an error. To suppress it, clip at a large finite value such as `-100.0 / 100.0` for the price-return dimensions.

“The buy-and-hold curve is trending upward over the episode.”

Check your random seed handling. If the same episode is being repeated (seed not reset), a lucky price trend will make buy-and-hold look profitable. Make sure `env.reset()` is called without a fixed seed in `average_cumulative_pnl`.

“My transaction cost makes the agent always hold flat (position = 0).”

That can happen if `transaction_cost` is too large relative to the reward scale. Try lowering it to 0.05 or 0.01 and see if the agent starts trading again.